WEEK 5: SUPERMARKET MANAGEMENT SYSTEM

Logeshwari D(E25AI121)

1. Scenario

The Supermarket Management System needs a system to manage payment transactions for customer sales. The system stores payment details such as payment date, payment mode, payment status, and transaction amount.

The system helps the supermarket maintain payment information, track successful and failed transactions, analyze payment methods, and generate payment-related business reports.

The Payment module is connected with the existing **CUSTOMER and SALES** tables from Week 4.

---

2. Payment Table

The PAYMENT table stores information about payment transactions made by customers for their sales.

Attribute	Description
PAYMENT_ID	Unique payment identifier
SALE_ID	Sale reference
PAYMENT_DATE	Date of payment
PAYMENT_MODE	UPI / Card / Cash
PAYMENT_STATUS	Success / Failed / Pending
TRANSACTION_AMOUNT	Amount paid

```
SQL> CREATE TABLE PAYMENT (
  2  PAYMENT_ID VARCHAR2(10) PRIMARY KEY,
  3  SALE_ID VARCHAR2(10) NOT NULL,
  4  PAYMENT_DATE DATE NOT NULL,
  5  PAYMENT_MODE VARCHAR2(20) NOT NULL,
  6  PAYMENT_STATUS VARCHAR2(20) NOT NULL,
  7  TRANSACTION_AMOUNT NUMBER(10,2) NOT NULL,
  8  CONSTRAINT FK_PAY_SALE_W5 FOREIGN KEY (SALE_ID) REFERENCES SALES(SALE_ID),
  9  CONSTRAINT CK_PAY_MODE_W5 CHECK (PAYMENT_MODE IN ('UPI','CARD','CASH')),
 10  CONSTRAINT CK_PAY_STATUS_W5 CHECK (PAYMENT_STATUS IN ('SUCCESS','FAILED','PENDING')),
 11  CONSTRAINT CK_PAY_AMOUNT_W5 CHECK (TRANSACTION_AMOUNT > 0)
 12  );

Table created.
```

---

3. Establish Relationship Using Primary Key and Foreign Key

The PAYMENT_ID uniquely identifies each payment transaction.

The SALE_ID in the PAYMENT table is used as a **Foreign Key** and connects the payment with the existing SALES table from Week 4.

SALES

|

| SALE_ID

↓

PAYMENT

One sale can have a payment transaction associated with it.

---

4. Apply Constraints

The following constraints are applied to maintain data accuracy:

Primary Key

- PAYMENT_ID uniquely identifies every payment.

Foreign Key

- SALE_ID in PAYMENT references SALES(SALE_ID).

NOT NULL

Important fields such as:

- PAYMENT_ID
- SALE_ID
- PAYMENT_DATE
- PAYMENT_MODE
- PAYMENT_STATUS
- TRANSACTION_AMOUNT

should not be left empty.

CHECK Constraint

The payment mode is restricted to:

- UPI
- Card
- Cash

The payment status is restricted to:

- Success
- Failed
- Pending

The transaction amount must be greater than zero.

```
SQL> DESC PAYMENT;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 PAYMENT_ID                                NOT NULL VARCHAR2(10)
 SALE_ID                                   NOT NULL VARCHAR2(10)
 PAYMENT_DATE                              NOT NULL DATE
 PAYMENT_MODE                              NOT NULL VARCHAR2(20)
 PAYMENT_STATUS                            NOT NULL VARCHAR2(20)
 TRANSACTION_AMOUNT                        NOT NULL NUMBER(10,2)
```

---

5. Inserting Payment Details

Sample payment records are inserted into the PAYMENT table.

The records contain:

- Payment ID
- Sale ID
- Payment date
- Payment mode
- Payment status
- Transaction amount

After insertion, the contents of the PAYMENT table are displayed to verify that the data has been added successfully.

```
SQL> SELECT * FROM PAYMENT;

PAYMENT_ID SALE_ID    PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
P001       S001       01-SEP-26 UPI                  SUCCESS
             415

P002       S002       02-SEP-26 CARD                 SUCCESS
             300

P003       S003       03-SEP-26 CASH                 SUCCESS
             340


PAYMENT_ID SALE_ID    PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
P004       S004       04-SEP-26 UPI                  FAILED
             470

P005       S005       05-SEP-26 CARD                 PENDING
             360
```

---

6. Manage Payment Transactions

The following operations are performed:

1. Store payment details for customer sales.
2. Display successful payments.
3. Find failed transactions.
4. Count successful and failed payments.
5. Analyze payment methods.
6. Calculate total revenue.
7. Calculate revenue by payment method.
8. Calculate average transaction amount.
9. Display customer payment history.

---

7. Display Successful Payments

This report displays all payment transactions whose payment status is **Success**.

Information displayed:

- Payment ID
- Sale ID
- Payment Date

- Payment Mode
- Transaction Amount

```
SQL> SELECT *
  2  FROM PAYMENT
  3  WHERE PAYMENT_STATUS = 'SUCCESS';

PAYMENT_ID SALE_ID    PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- ---------------
TRANSACTION_AMOUNT
------------------
P001       S001       01-SEP-26 UPI                  SUCCESS
               415

P002       S002       02-SEP-26 CARD                 SUCCESS
               300

P003       S003       03-SEP-26 CASH                 SUCCESS
               340
```

---

8. Find Failed Transactions

This report identifies payment transactions whose status is **Failed**.

It helps the supermarket track unsuccessful payment transactions.

Information displayed:

- Payment ID
- Sale ID
- Payment Date
- Payment Mode
- Transaction Amount
- Payment Status

```
SQL> SELECT *
  2  FROM PAYMENT
  3  WHERE PAYMENT_STATUS = 'FAILED';

PAYMENT_ID SALE_ID    PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- ---------------
TRANSACTION_AMOUNT
------------------
P004       S004       04-SEP-26 UPI                  FAILED
               470
```

---

9. Count Successful and Failed Payments

The COUNT() function and GROUP BY clause are used to calculate the number of successful and failed payment transactions.

The report displays:

- Payment Status
- Number of Transactions

```
SQL> SELECT PAYMENT_STATUS, COUNT(*) AS TOTAL_PAYMENTS
  2  FROM PAYMENT
  3  WHERE PAYMENT_STATUS IN ('SUCCESS', 'FAILED')
  4  GROUP BY PAYMENT_STATUS;

PAYMENT_STATUS       TOTAL_PAYMENTS
-------------------- --------------
FAILED                            1
SUCCESS                           3
```

---

10. Payment Mode Analysis

This report analyzes the different payment methods used by customers.

To Find:

- Number of transactions using UPI
- Number of Card payments
- Number of Cash payments
- Most preferred payment method

The COUNT() and GROUP BY functions are used to analyze payment modes.

```
SQL> SELECT PAYMENT_MODE, COUNT(*) AS TOTAL_PAYMENTS
  2  FROM PAYMENT
  3  GROUP BY PAYMENT_MODE
  4  HAVING COUNT(*) = (
  5      SELECT MAX(COUNT(*))
  6      FROM PAYMENT
  7      GROUP BY PAYMENT_MODE
  8  );

PAYMENT_MODE         TOTAL_PAYMENTS
-------------------- --------------
CARD                              2
UPI                               2
```

---

11. Revenue Analysis

This report analyzes the revenue generated through successful payment transactions.

To Find:

- Total revenue generated
- Revenue by payment method
- Average transaction amount

The SUM() function is used to calculate total revenue, while AVG() is used to calculate the average transaction amount.

Only successful payments are considered for revenue calculation.

```
SQL> SELECT SUM(TRANSACTION_AMOUNT) AS TOTAL_REVENUE
  2  FROM PAYMENT
  3  WHERE PAYMENT_STATUS = 'SUCCESS';

TOTAL_REVENUE
-------------
         1055
```

```
SQL> SELECT
  2      PAYMENT_MODE,
  3      SUM(TRANSACTION_AMOUNT) AS TOTAL_REVENUE
  4  FROM PAYMENT
  5  WHERE PAYMENT_STATUS = 'SUCCESS'
  6  GROUP BY PAYMENT_MODE
  7  ORDER BY TOTAL_REVENUE DESC;

PAYMENT_MODE         TOTAL_REVENUE
-------------------- -------------
UPI                            415
CASH                           340
CARD                           300

SQL> SELECT AVG(TRANSACTION_AMOUNT) AS AVERAGE_TRANSACTION_AMOUNT
  2  FROM PAYMENT
  3  WHERE PAYMENT_STATUS = 'SUCCESS';

AVERAGE_TRANSACTION_AMOUNT
--------------------------
                351.666667
```

---

12. Customer Payment History

This report displays the payment history of customers by joining the existing CUSTOMER, SALES, and PAYMENT tables.

Information displayed:

- Customer Name
- Sale ID

- Payment Mode
- Transaction Amount
- Payment Status

The JOIN operation is used to combine the customer, sales, and payment information Week 4 already uses the CUSTOMER and SALES relationship for customer purchase history, so this extends that existing structure.

```
SQL> SELECT
  2      C.CUSTOMER_NAME,
  3      S.SALE_ID,
  4      P.PAYMENT_MODE,
  5      P.TRANSACTION_AMOUNT,
  6      P.PAYMENT_STATUS
  7  FROM CUSTOMER C
  8  JOIN SALES S
  9      ON C.CUSTOMER_ID = S.CUSTOMER_ID
 10  JOIN PAYMENT P
 11      ON S.SALE_ID = P.SALE_ID;

CUSTOMER_NAME                                      SALE_ID
-------------------------------------------------- ----------
PAYMENT_MODE         TRANSACTION_AMOUNT PAYMENT_STATUS
-------------------- ------------------ --------------------
Arun                                               S001
UPI                                 415 SUCCESS

Priya                                              S002
CARD                                300 SUCCESS

Karthik                                            S003
CASH                                340 SUCCESS


CUSTOMER_NAME                                      SALE_ID
-------------------------------------------------- ----------
PAYMENT_MODE         TRANSACTION_AMOUNT PAYMENT_STATUS
-------------------- ------------------ --------------------
Divya                                              S004
UPI                                 470 FAILED

Rahul                                              S005
CARD                                360 PENDING
```

---

13. Payment Analysis

The Payment Analysis report provides important information about supermarket transactions.

To Find:

Total Successful Payments
The total number of successfully completed payment transactions.

Total Failed Payments
The total number of failed payment transactions.

Most Preferred Payment Method
The payment method used most frequently by customers.

Total Revenue
The total amount generated from successful payments.

Average Transaction Amount
The average amount paid per successful transaction.

These analyses follow the payment-analysis requirements given in your mam's Week 5 project.

```
SQL> SELECT
  2      COUNT(CASE WHEN PAYMENT_STATUS = 'SUCCESS' THEN 1 END) AS TOTAL_SUCCESSFUL_PAYMENTS,
  3      COUNT(CASE WHEN PAYMENT_STATUS = 'FAILED' THEN 1 END) AS TOTAL_FAILED_PAYMENTS,
  4      SUM(CASE WHEN PAYMENT_STATUS = 'SUCCESS' THEN TRANSACTION_AMOUNT ELSE 0 END) AS TOTAL_REVENUE,
  5      AVG(CASE WHEN PAYMENT_STATUS = 'SUCCESS' THEN TRANSACTION_AMOUNT END) AS AVG_TRANSACTION_AMOUNT
  6  FROM PAYMENT;

TOTAL_SUCCESSFUL_PAYMENTS TOTAL_FAILED_PAYMENTS TOTAL_REVENUE
------------------------- --------------------- -------------
AVG_TRANSACTION_AMOUNT
----------------------
                        3                     1          1055
           351.666667
```

---

14. Conclusion

The Week 5 Supermarket Management System extends the existing product, category, supplier, inventory, customer, and sales management modules by adding payment transaction management.

The PAYMENT table is connected with the existing SALES table using primary key and foreign key relationships. The system stores payment details, manages successful and failed transactions, and analyzes different payment methods.

Various reports are generated, including successful payments, failed transactions, payment mode analysis, revenue analysis, and customer payment history.

Thus, the system provides an organized way to manage supermarket payment transactions and analyze customer payment information.